

PROJECT REFERENCE MANUAL

EinsteinAI

Architecture, API, database, development and operations

DOCUMENT SCOPE

Current implementation reference for the React, FastAPI, LangChain, Gemini and Supabase application.

Prepared from the project source and SQL setup scripts
Version: 1.0 | 29 September 2026

Contents

Product capabilities	3
Architecture at a glance	3
Development and operations	4
System requirements	4
Environment configuration	4
Install dependencies	4
Run the application	5
Request lifecycle	5
Logs and health checks	5
Production readiness checklist	6
Scope and limitations	6
EinsteinAI HTTP API	7
Base URL	7
Authentication	7
GET /health	7
GET /api/usage	7
POST /api/chat	7
Error responses	8
CORS	8
Supabase database and plan administration	9
Apply the SQL setup	9
Main data	9
Security model	10
Question allowance behavior	10
Assign a plan by email	11
Verify the assigned plan	12
Operational notes	12
Subject Index	13

EinsteinAI is a web-based physics tutor. It combines a React and Vite client, a FastAPI service, a LangChain agent powered by Google Gemini, and Supabase for authentication, conversation storage, and account usage.

This repository is a local-development project. The guide below documents the current implementation and its manual plan management; it does not imply that payments or production hosting are configured.

Product capabilities

- Account registration, sign-in, sign-out, and editable display name through Supabase Auth.
- Physics-focused answers generated by a LangChain agent using Google Gemini.
- Optional step-by-step solutions for physics problems.
- Conversation history saved to Supabase, with conversation search, pinning, and deletion.
- Message search within a conversation; copy answers; export as text or print/save as PDF.
- Edit and resend a user question. The edited question replaces the original and removes later messages in that conversation before receiving a new answer.
- Suggested follow-up prompts to continue exploring a topic.
- Profile page with plan status, expiry, question balance, and model token usage.
- Manually administered access: two free questions per account, a 10-question pack, or unlimited monthly/yearly subscriptions. There is no payment checkout.

Architecture at a glance

```
React browser app
- Supabase Auth ----- user identity and session
- Supabase database ----- profile, conversations, messages, usage
- FastAPI backend ----- bearer-token verification and quota enforcement
  - LangChain agent ----- conversation context and calculator tool
  - Google Gemini ----- physics answer generation
```

The browser sends its Supabase access token to the backend. The backend validates that token, applies the user's allowance through server-only database functions, calls Gemini, and returns the answer and usage data. The Supabase service-role key and Gemini key stay on the backend.

Development and operations

This document covers setup and local operation of EinsteinAI. The repository currently describes a development configuration; complete the deployment checklist before exposing it to public traffic.

System requirements

- Python 3.10 or newer.
- Node.js and npm compatible with the Vite version installed in `frontend/`.
- Supabase project with the repository SQL setup applied.
- Google Gemini API key accepted by the configured Gemini model.

A root `.venv` is assumed by the run commands below. If the virtual environment is elsewhere, replace `../.venv/bin/python` with that environment's Python executable.

Environment configuration

Backend: `backend/.env`

Start by copying `backend/.env.example` to `backend/.env` and fill in values:

Variable	Required	Description
GEMINI_API_KEY	Yes	Private Google Gemini API key used by LangChain.
SUPABASE_URL	Yes for authenticated API calls	Project URL, e.g. <code>https://<project-ref>.supabase.co</code> .
SUPABASE_PUBLISHABLE_KEY	Yes for authenticated API calls	Public/publishable Supabase key used to verify user sessions. <code>SUPABASE_ANON_KEY</code> is accepted as a compatibility fallback in code.
SUPABASE_SERVICE_ROLE_KEY	Yes for authenticated API calls	Server-only key for protected usage operations. Never expose to the browser.
CORS_ORIGINS	No	Comma-separated trusted frontend origins. Defaults include localhost Vite ports.

Frontend: `frontend/.env`

Variable	Required	Description
VITE_SUPABASE_URL	Yes	Supabase project URL.
VITE_SUPABASE_PUBLISHABLE_KEY	Yes	Browser-safe Supabase publishable key.
VITE_API_URL	Yes	API base URL; local default is <code>http://localhost:8000</code> .

Vite exposes every `VITE_*` value to browser code. Only put values intended to be public in this file. Do not commit local `.env` files. After editing backend settings, restart Uvicorn; after editing frontend settings, restart Vite.

Install dependencies

From the repository root:

```
cd backend
../.venv/bin/python -m pip install -r requirements.txt
```

In another terminal:

```
cd frontend
npm install
```

The frontend package includes build, lint, and preview scripts. The application instructions should use the npm scripts as defined in `frontend/package.json`.

Run the application

Backend terminal:

```
cd backend
../.venv/bin/python -m uvicorn main:app --reload --port 8000
```

Frontend terminal:

```
cd frontend
npm run dev
```

Use the URL Vite prints. If it selects port 5174 because 5173 is occupied, that local origin is included in the backend's default CORS configuration.

Request lifecycle

- 1 The user signs in through Supabase Auth in the browser.
- 2 The React app sends the current Supabase access token in the `Authorization: Bearer ...` header to protected API routes.
- 3 FastAPI verifies the token and resolves the authenticated user ID.
- 4 For chat requests, the backend calls the database quota function before contacting Gemini.
- 5 LangChain invokes the Gemini model and the arithmetic tool when needed.
- 6 The backend records token usage, returns the response, and attempts to restore consumed allowance if generation fails.
- 7 The frontend stores and displays chat history through Supabase under the signed-in user's identity.

Logs and health checks

- `GET /health` confirms the API process is responding; it does not test Supabase or Gemini credentials.
- Uvicorn logs request method, path, status, and elapsed milliseconds.
- Exceptions from Supabase or LangChain are logged on the backend. User-facing API errors are intentionally more general.
- Do not share complete terminal logs if they may contain user information, request content, environment values, or tokens. The current request middleware logs paths and status rather than authorization headers.

For a first diagnosis, check in this order:

- 1 Open `/health` and confirm the API responds.
- 2 Confirm `VITE_API_URL` points to the backend address and restart Vite after changes.
- 3 Confirm `backend/.env` has the correct Gemini and Supabase settings; restart Uvicorn.
- 4 Confirm `CORS_ORIGINS` exactly matches the page origin, including port.

- 5 Confirm all SQL scripts ran in the same Supabase project as the configured URL and keys.
- 6 Inspect the backend traceback for the specific Supabase or Gemini failure.

Production readiness checklist

Before making the app publicly available:

- Use managed secret storage for backend keys; rotate any key that was committed or shared.
- Configure exact production CORS origins and HTTPS for both frontend and backend.
- Do not expose a service-role key to client code, static hosting configuration, logs, or source maps.
- Review RLS policies on conversations, messages, profile data, and any other user-scoped tables; test that one account cannot read or change another account's data.
- Add rate limiting and abuse controls for the chat endpoint.
- Decide retention, export, deletion, and backup policies for user conversations.
- Add monitored health checks and alerts for API errors, Gemini availability, quota RPC failures, and database connectivity.
- Define how manual plan assignments are authorized, audited, extended, and revoked.
- Add billing only if desired; current plan flags are administrative and are not proof of payment.
- Review provider quotas, privacy terms, and retention settings for the Gemini and Supabase projects.
- Run build, lint, and integration checks in the target deployment environment before release.

Scope and limitations

- No payment provider or checkout flow is wired in.
- Plan changes depend on a trusted administrator editing `public.account_usage`.
- The health route is process-level only; it does not provide a dependency readiness check.
- The assistant is prompted to answer physics questions and can still produce incorrect explanations. The interface advises users to check important physics answers.

EinsteinAI HTTP API

This guide describes the FastAPI service currently implemented in `backend/main.py`.

Base URL

Local development uses `http://localhost:8000`. FastAPI's interactive API documentation is available at `/docs`; its OpenAPI schema is at `/openapi.json`.

Authentication

The protected endpoints require the Supabase access token for the signed-in user:

```
Authorization: Bearer <supabase-access-token>
```

The backend validates the token with Supabase Auth before using the authenticated user ID for database operations. Never accept a user ID supplied by the browser as an authorization decision. The backend's service-role client is only created after token validation and remains server-side.

GET /health

Simple process health endpoint. It does not check connectivity to Supabase or Gemini.

Example response:

```
{"status": "ok"}
```

GET /api/usage

Returns the authenticated user's `account_usage` record. Requires the bearer token.

Fields include:

Field	Meaning
<code>questions_used</code>	Free questions used by this account.
<code>question_credits</code>	Remaining purchased/manual question credits.
<code>input_tokens</code>	Accumulated model input-token count.
<code>output_tokens</code>	Accumulated model output-token count.
<code>subscription_status</code>	free, active, canceling, canceled, or past_due.
<code>subscription_ends_at</code>	Subscription expiry timestamp, or null.
<code>created_at, updated_at</code>	Record timestamps.

The frontend also reads `plan_type` and `question_credits_expires_at` directly from Supabase under the configured user-level RLS policies.

POST /api/chat

Submits a physics question. Requires the bearer token and a JSON body:

```
{
  "question": "Why does time slow down near a black hole?",
  "history": [
    {"role": "user", "content": "What is gravity?"},
  ]
}
```

```
{
  "role": "assistant", "content": "Gravity is ..."
},
"show_steps": false
}
```

Validation limits:

- question: 1-4,000 characters.
- history: at most 12 messages.
- Each message role must be user or assistant.
- Each message content: 1-12,000 characters.
- show_steps: boolean; when true, the agent is instructed to structure numerical solutions step by step.

Successful response:

```
{
  "answer": "A clear explanation...",
  "input_tokens": 123,
  "output_tokens": 245,
  "questions_used": 2,
  "questions_remaining": 0,
  "subscription_status": "free"
}
```

questions_remaining is null for an active unlimited subscription. The backend consumes an allowance before model generation and attempts to refund it if generation fails. Token usage is recorded separately; a failure to record token counts does not discard a successful answer.

Error responses

Errors use FastAPI's JSON detail format. Common status codes:

Status	Meaning
401	Missing, invalid, or expired Supabase access token.
402	No free questions or valid question credits remain; an active plan is not available.
422	Request body failed validation.
500	A database usage or quota operation failed. Check backend logs.
502	Gemini/LangChain generation failed. The backend attempts to restore the consumed question/credit.
503	Required backend Supabase configuration is incomplete.

CORS

The backend reads comma-separated origins from CORS_ORIGINS in backend/.env. It currently also adds `http://localhost:5173` and `http://localhost:5174` for Vite development. For a hosted deployment, configure the exact trusted frontend origin(s); do not use a wildcard for a credentialed production app.

Supabase database and plan administration

EinsteinAI uses Supabase Auth for identity and Postgres for conversations, messages, and usage. Database setup is maintained as SQL files in `supabase/` and should be applied to the same Supabase project whose URL and keys are configured in the app.

Apply the SQL setup

Use Supabase Dashboard -> SQL Editor. If this is a new database and the chat tables do not exist yet, first run the reference schema in Chat table prerequisites. Then apply the included scripts in this order:

- 1 `supabase/subscription_usage.sql`
- 2 `supabase/manual_plans_setup.sql`
- 3 `supabase/chat_features.sql`

The scripts use `create ... if not exists`, `create or replace`, and `drop ... if exists` in key places, but they are not a general migration framework. Keep a copy of the database state before making substantial schema changes, and apply future changes as new, versioned migrations rather than editing already-applied history without a plan.

Main data

`public.account_usage`

One row per Supabase Auth user. A signup trigger creates the row, and the base migration backfills rows for existing users.

Column	Purpose
<code>user_id</code>	Primary key and reference to <code>auth.users.id</code> .
<code>questions_used</code>	Count of free questions consumed. It is not reset automatically by plan assignment.
<code>question_credits</code>	Available 10-pack credits.
<code>question_credits_expires_at</code>	End-of-day expiry for question credits.
<code>plan_type</code>	<code>free</code> , <code>questions_10</code> , <code>monthly</code> , or <code>yearly</code> .
<code>subscription_status</code>	Current subscription access state.
<code>subscription_ends_at</code>	Subscription expiry timestamp.
<code>input_tokens</code> , <code>output_tokens</code>	Accumulated model token counts.
<code>created_at</code> , <code>updated_at</code>	Record timestamps.

The application also uses `public.conversations` and `public.messages` for saved chats. The three included SQL files do not create these two chat tables. If they are not already present in your project, create them before using chat history. The frontend expects the columns shown below and relies on row-level security (RLS) to isolate accounts.

Chat table prerequisites for a new database

The following is a reference setup for projects that do not already have these tables. Review it against any existing schema before running it; do not create duplicate or conflicting tables. Run it before `chat_features.sql` so the latter can add the `pin` column.

```
create table if not exists public.conversations (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
```

```

    title text not null default 'New conversation',
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    is_pinned boolean not null default false,
    unique (id, user_id)
);
create table if not exists public.messages (
    id uuid primary key default gen_random_uuid(),
    conversation_id uuid not null,
    user_id uuid not null references auth.users(id) on delete cascade,
    role text not null check (role in ('user', 'assistant')),
    content text not null,
    created_at timestamptz not null default now(),
    foreign key (conversation_id, user_id)
        references public.conversations(id, user_id) on delete cascade
);
alter table public.conversations enable row level security;
alter table public.messages enable row level security;
revoke all on public.conversations, public.messages from anon;
grant select, insert, update, delete on public.conversations, public.messages to authenticated;
drop policy if exists "Users manage their own conversations" on public.conversations;
create policy "Users manage their own conversations"
    on public.conversations for all to authenticated
    using ((select auth.uid()) = user_id)
    with check ((select auth.uid()) = user_id);
drop policy if exists "Users manage their own messages" on public.messages;
create policy "Users manage their own messages"
    on public.messages for all to authenticated
    using ((select auth.uid()) = user_id)
    with check ((select auth.uid()) = user_id);

```

This composite foreign key ensures a message's `user_id` matches the owner of its conversation. The app currently updates conversation titles and timestamps from the browser, so the conversation policy allows the owner to update their own row. If these tables already exist, verify that the required columns and equivalent RLS protections are present instead of applying this example verbatim.

Security model

- RLS is enabled for `account_usage`; authenticated users may read their own row only.
- Browser users do not receive write access to the usage table or execute permissions on quota/token RPC functions.
- The backend validates the incoming Supabase access token, obtains the verified user ID, and performs protected operations with its server-side service-role client.
- Keep the service-role key and Gemini key secret. Do not put either value in a Vite variable or in frontend source.
- Administrative plan changes should be performed only by a trusted project administrator in Supabase SQL Editor.

Question allowance behavior

`consume_question` serializes account updates with a row lock. If an active subscription has not expired, the request is allowed without a question limit. Otherwise, the user can use up to two free questions, followed by valid question credits. Expired credits are cleared when the quota function runs. The backend attempts to refund a consumed free question or pack credit if model generation fails.

In the current SQL implementation, every request made while a subscription is active also increments `questions_used`. Plan assignment does not reset that counter. Consequently, a later return to free access can inherit the accumulated counter. This is the behavior of the current code and should be revisited if subscription questions are intended to leave the free allowance untouched.

Token totals come from model response usage metadata and are stored through `record_token_usage`. They are estimates of model usage reported by the provider, not a billing ledger.

Assign a plan by email

Replace the email below with the exact account email. Each query updates the row for that Supabase Auth user. If the account row does not exist yet, have the user register first; the signup trigger creates it.

Monthly subscription

```
UPDATE public.account_usage
SET plan_type = 'monthly'
WHERE user_id = (
  SELECT id FROM auth.users
  WHERE email = lower('person@example.com')
);
```

The trigger sets `subscription_status = 'active'`, sets expiry to one month from the update, and clears question credits. Reassigning the monthly plan starts a fresh one-month period.

Yearly subscription

```
UPDATE public.account_usage
SET plan_type = 'yearly'
WHERE user_id = (
  SELECT id FROM auth.users
  WHERE email = lower('person@example.com')
);
```

The trigger sets the subscription to active for one year from the update and clears question credits. Reassigning the yearly plan starts a fresh one-year period.

Add a 10-question pack

```
UPDATE public.account_usage
SET plan_type = 'questions_10'
WHERE user_id = (
  SELECT id FROM auth.users
  WHERE email = lower('person@example.com')
);
```

This adds 10 credits to any still-valid credits and sets expiry to the next midnight in Asia/Kolkata. Reapplying this plan adds another 10 credits and refreshes the end-of-day expiry.

Return to free

```
UPDATE public.account_usage
SET plan_type = 'free'
WHERE user_id = (
  SELECT id FROM auth.users
  WHERE email = lower('person@example.com')
);
```

This clears subscription access and question credits. It does not reset `questions_used`; the free two-question allowance is account-lifetime under the current implementation.

Verify the assigned plan

```
SELECT
  u.email,
  a.plan_type,
  a.subscription_status,
  a.subscription_ends_at,
  a.question_credits,
  a.question_credits_expires_at,
  a.questions_used,
  a.input_tokens,
  a.output_tokens
FROM public.account_usage AS a
JOIN auth.users AS u ON u.id = a.user_id
WHERE u.email = lower('person@example.com');
```

Refresh the profile page after changing a plan. The profile reads the database values and derives whether a subscription is currently active based on its status and expiry.

Operational notes

- These plan changes are manual administrative assignments. The app does not collect money, verify payment, create invoices, renew subscriptions, or notify users by email.
- A monthly/yearly assignment changes the expiry relative to the time the SQL update runs, not a date entered by the admin.
- An expired subscription is no longer treated as active. Its historical `plan_type` may remain monthly/yearly until an administrator changes it; access is determined from status and expiry.
- Assigning the question pack marks `subscription_status` as `free`; the app recognizes the pack from `question_credits` and its expiry.

Subject Index

API authentication	7	Manual plans	11
API chat endpoint	7	Monthly plan	11
API errors	8	Question credits	11
CORS	8	Question allowance	10
Database migrations	9	RLS and access policies	10
Environment variables	4	Troubleshooting	5
Gemini	6	Token usage	10
Health checks	7	Vite startup	5
LangChain	3	Yearly plan	11